

2. ОСНОВЫ (FOUNDATIONS)

Погружаясь в более продвинутые уголки Rust, важно убедиться, что у вас есть прочное понимание фундаментальных вещей. В Rust, как и в любом языке программирования, точный смысл различных ключевых слов и концепций становится важным по мере того, как вы начинаете использовать язык более изощрёнными способами. В этой главе мы пройдемся по многим примитивам Rust и постараемся более ясно определить, что они означают, как работают и почему они именно такие. В частности, мы рассмотрим, чем различаются переменные и значения, как они представлены в памяти и какие области памяти есть у программы. Затем мы обсудим некоторые тонкости владения (ownership), заимствования (borrowing) и времён жизни (lifetimes), которые вам нужно усвоить, прежде чем продолжить чтение книги.

Эту главу можно читать сверху вниз, если хотите, либо использовать как справочник для освежения концепций, в которых вы чувствуете себя менее уверенно. Я рекомендую двигаться дальше только тогда, когда вы чувствуете себя полностью уверенно с содержанием этой главы, поскольку заблуждения о том, как работают эти примитивы, быстро встанут на пути понимания более продвинутых тем или приведут к их неправильному использованию.

Разговор о памяти

Не вся память создана равной. В большинстве сред программирования ваши программы имеют доступ к стеку (stack), куче (heap), регистрам (registers), сегментам кода (text segments), регистрам, отображённым в память (memory-mapped registers), файлам, отображённым в память (memory-mapped files), и, возможно, к энергонезависимой оперативной памяти (nonvolatile RAM). То, что именно вы выберете использовать в конкретной ситуации, влияет на то, что вы можете там хранить, как долго это остаётся доступным и какие механизмы вы используете для доступа к этому. Точные детали этих областей памяти варьируются между платформами и выходят за рамки этой книги, но некоторые из них настолько важны для того, как вы рассуждаете о коде на Rust, что заслуживают рассмотрения здесь.

Терминология памяти

Прежде чем погрузиться в области памяти, вам сначала нужно узнать о различии между значениями (values), переменными (variables) и указателями (pointers). *Значение* в Rust — это комбинация типа и элемента из области значений этого типа. Значение можно превратить в последовательность байтов с помощью *представления* (representation) его типа, но само по себе значение лучше воспринимать скорее как то, что имели в виду вы, программист. Например, число `6` типа `u8` является экземпляром математического целого числа 6, а его представление в памяти — это байт `0x06`. Аналогично, строка `str "Hello world"` — это значение из области всех строк, чьё представление — её UTF-8-кодировка. Смысл значения независим от того, где хранятся эти байты.

Значение хранится в *месте* (place) — это терминология Rust для «локации, которая может хранить значение». Это место может быть на стеке, на куче или в ряде других мест. Наиболее распространённое место для хранения значения — это *переменная* (variable), которая представляет собой именованный слот для значения на стеке.

Указатель (pointer) — это значение, которое содержит адрес области памяти, так что указатель указывает на место. Указатель можно разыменовать (dereference), чтобы получить доступ к значению, хранящемуся в локации памяти, на которую он указывает. Мы можем хранить один и тот же указатель в более чем одной переменной и, таким образом, иметь несколько переменных, косвенно ссылающихся на одну и ту же локацию в памяти, а значит и на одно и то же лежащее в основе значение.

Рассмотрим код в листинге 2-1, который иллюстрирует эти три элемента.

```
let x = 42;
let y = 43;
let var1 = &x;
let mut var2 = &x;
❶ var2 = &y;
```

Листинг 2-1: Значения, переменные и указатели

Здесь есть четыре различных значения: `42` (типа `i32`), `43` (типа `i32`), адрес `x` (указатель) и адрес `y` (указатель). Также есть четыре переменные: `x`, `y`, `var1` и `var2`. Две последние переменные обе хранят значения типа указатель, потому что ссылки являются указателями. Хотя `var1` и `var2` изначально хранят одно и

то же значение, они хранят отдельные, независимые копии этого значения; когда мы меняем значение, хранящееся в `var2` ❶, значение в `var1` не меняется. В частности, оператор `=` сохраняет значение выражения с правой стороны в месте, названном с левой стороны.

Интересный пример того, где различие между переменными, значениями и указателями становится важным, — это выражение вроде:

```
let string = "Hello world";
```

Хотя мы присваиваем строковое значение переменной `string`, *фактическое* значение этой переменной — это указатель на первый символ в строковом значении `"Hello world"`, а не сама строка. На этом этапе вы можете сказать: «Погодите, а где же тогда хранится строковое значение? Куда указывает указатель?» Если так, у вас зоркий глаз — мы доберёмся до этого через секунду.

ПРИМЕЧАНИЕ Технически значение `string` также включает в себя длину строки. Мы поговорим об этом в главе 3, когда будем обсуждать типы «толстых указателей» (fat pointer types).

Переменные подробнее

Определение переменной, которое я дал ранее, является широким и вряд ли особо полезным само по себе. По мере того как вы будете встречать более сложный код, вам понадобится более точная ментальная модель, чтобы рассуждать о том, что программы на самом деле делают. Существует много таких моделей, которые мы можем использовать. Описание их всех в деталях заняло бы несколько глав и выходит за рамки этой книги, но в широком смысле их можно разделить на две категории: высокоуровневые модели и низкоуровневые модели. Высокоуровневые модели полезны, когда вы думаете о коде на уровне времён жизни и заимствований, тогда как низкоуровневые модели хороши, когда вы рассуждаете о небезопасном (unsafe) коде и сырых указателях. Моделей для переменных, описанных в следующих двух разделах, будет достаточно для большей части материала этой книги.

Высокоуровневая модель

В высокоуровневой модели мы не думаем о переменных как о местах, которые хранят байты. Вместо этого мы думаем о них просто как об именах, данных

значениям по мере того, как они создаются, перемещаются и используются на протяжении программы. Когда вы присваиваете значение переменной, это значение с этого момента называется этой переменной. Когда переменная позже используется, можно вообразить линию, проведенную от предыдущего обращения к этой переменной к новому обращению, которая устанавливает зависимостное отношение между двумя обращениями. Если значение в переменной перемещено (moved), от неё больше нельзя проводить линии.

В этой модели переменная существует только до тех пор, пока она хранит допустимое значение; нельзя провести линии от переменной, чьё значение неинициализировано или было перемещено, так что фактически её нет. Используя эту модель, вся ваша программа состоит из множества таких зависимостных линий, часто называемых *потоками* (flows), каждая из которых отслеживает время жизни конкретного экземпляра значения. Потоки могут разветвляться и сливаться, когда есть ветвления, при этом каждое ветвление образует отдельное время жизни для этого значения. Компилятор может проверить, что в любой данной точке вашей программы все потоки, которые могут существовать параллельно друг с другом, совместимы. Например, не может быть двух параллельных потоков с изменяемым (mutable) доступом к значению. Также не может быть потока, который заимствует значение, в то время как нет потока, владеющего этим значением. Листинг 2-2 показывает примеры обоих этих случаев.

```
let mut x;  
// это обращение было бы недопустимым, неоткуда провести поток:  
// assert_eq!(x, 42);  
❶ x = 42;  
// это нормально, можно провести поток от значения, присвоенного выше:  
❷ let y = &x;  
// это устанавливает второй, изменяемый поток от x:  
❸ x = 43;  
// это продолжает поток от y, который, в свою очередь, исходит от x,  
// но этот поток конфликтует с присваиванием x!  
❹ assert_eq!(*y, 42);
```

Листинг 2-2: Недопустимые потоки, которые перехватит borrow checker

Сначала мы не можем использовать `x`, прежде чем он будет инициализирован, потому что нам неоткуда провести поток. Только когда мы присваиваем значение `x`, мы можем проводить от него потоки. В этом коде есть два потока: один эксклюзивный (`&mut`) поток от ❶ к ❸ и один разделяемый (`&`) поток от ❶ через ❷ к ❹. Borrow checker проверяет каждую вершину каждого потока и проверяет, что не существует других несовместимых потоков одновременно. В

данном случае, когда borrow checker проверяет эксклюзивный поток на ❸, он видит разделяемый поток, который заканчивается на ❹. Поскольку нельзя одновременно иметь эксклюзивное и разделяемое использование значения, borrow checker (корректно) отвергает код. Заметьте, что если бы ❹ не было, этот код прекрасно скомпилировался бы! Разделяемый поток завершился бы на ❷, и когда эксклюзивный поток проверяется на ❸, никаких конфликтующих потоков не существовало бы.

Если новая переменная объявлена с тем же именем, что и предыдущая, они всё равно считаются различными переменными. Это называется *затенением* (shadowing) — более поздняя переменная «затеняет» более раннюю с тем же именем. Две переменные сосуществуют, хотя последующий код больше не имеет способа сослаться на более раннюю. Эта модель примерно соответствует тому, как компилятор и, в частности, borrow checker рассуждают о вашей программе, и фактически используется внутри компилятора для производства эффективного кода.

Низкоуровневая модель

Переменные именуют локации памяти, которые могут или не могут содержать допустимые значения. Можно думать о переменной как о «слоте значения» (value slot). Когда вы присваиваете ему, слот заполняется, а его старое значение (если оно было) сбрасывается (dropped) и заменяется. Когда вы обращаетесь к нему, компилятор проверяет, что слот не пуст, поскольку это означало бы, что переменная неинициализирована или её значение было перемещено. Указатель на переменную ссылается на память, лежащую в основе переменной, и может быть разыменован для получения её значения. Например, в выражении `let x: usize`, переменная `x` — это имя для области памяти на стеке, в которой есть место для значения размером с `usize`, хотя оно и не имеет чётко определённого значения (его слот пуст). Если вы присвоите значение этой переменной, например `x = 6`, то эта область памяти будет содержать биты, представляющие значение `6`. `&x` не меняется, когда вы присваиваете `x`. Если вы объявляете несколько переменных с одинаковым именем, они всё равно оказываются с разной памятью, лежащей в их основе. Эта модель соответствует модели памяти, используемой C и C++ и многими другими низкоуровневыми языками, и полезна, когда вам нужно рассуждать о памяти явно.

ПРИМЕЧАНИЕ В этом примере мы игнорируем регистры процессора и рассматриваем их как оптимизацию. В реальности компилятор может использовать регистр для размещения переменной вместо области памяти, если для этой переменной не нужен адрес памяти.

Вы можете обнаружить, что одна из этих моделей соответствует вашей предыдущей модели лучше, чем другая, но я призываю вас попытаться освоить обе. Они обе одинаково правомерны и обе являются упрощениями, как и любая полезная ментальная модель. Если вы способны рассматривать фрагмент кода с обеих этих точек зрения, вам будет гораздо легче работать со сложными сегментами кода и понимать, почему они компилируются и работают (или не работают) так, как вы ожидаете.

Области памяти

Теперь, когда вы освоились с тем, как мы говорим о памяти, нам нужно поговорить о том, что память на самом деле представляет собой. Существует много различных областей памяти, и, возможно, удивительно, что не все из них хранятся в DRAM вашего компьютера. То, какую часть памяти вы используете, значительно влияет на то, как вы пишете код. Три наиболее важными областями для целей написания кода на Rust являются стек, куча и статическая память.

Стек

Стек (stack) — это сегмент памяти, который ваша программа использует как рабочее пространство для вызовов функций. Каждый раз, когда вызывается функция, непрерывный кусок памяти, называемый *кадром* (frame), выделяется в верхней части стека. Около нижней части стека находится кадр для функции `main`, и по мере того как функции вызывают другие функции, дополнительные кадры помещаются на стек. Кадр функции содержит все переменные внутри этой функции, а также любые аргументы, которые функция принимает. Когда функция возвращается, её кадр стека освобождается.

Байты, составляющие значения локальных переменных функции, не стираются немедленно, но небезопасно обращаться к ним, поскольку они могли быть перезаписаны последующим вызовом функции, чей кадр перекрывается с освобождённым. И даже если они не были перезаписаны, они могут содержать значения, которые недопустимо использовать, например те, что были перемещены (moved) при возврате из функции.

Кадры стека и, что критически важно, тот факт, что они в конечном итоге исчезают, очень тесно связаны с понятием времён жизни в Rust. Любая переменная, хранящаяся в кадре на стеке, не может быть доступна после того, как этот кадр исчезает, поэтому любая ссылка на неё должна иметь время жизни не более долгое, чем время жизни кадра.

Куча

Куча (heap) — это пул памяти, который не привязан к текущему стеку вызовов программы. Значения в памяти кучи живут до тех пор, пока они явно не освобождены. Это полезно, когда вы хотите, чтобы значение жило за пределами времени жизни кадра текущей функции. Если это значение является возвращаемым значением функции, вызывающая функция может оставить на своём стеке некоторое место, в которое вызванная функция запишет это значение перед возвратом. Но если вы хотите, скажем, отправить это значение другому потоку, с которым текущий поток может вообще не иметь общих кадров стека, вы можете хранить его в куче.

Куча позволяет явно выделять непрерывные сегменты памяти. Когда вы это делаете, вы получаете указатель на начало этого сегмента. Этот сегмент памяти зарезервирован за вами до тех пор, пока вы позже не освободите его; этот процесс часто называют *освобождением* (freeing), по имени соответствующей функции в стандартной библиотеке C. Поскольку выделения из кучи не исчезают, когда функция возвращается, вы можете выделить память для значения в одном месте, передать указатель на него в другой поток и позволить этому потоку безопасно продолжать работать с этим значением. Или, иначе говоря, когда вы выделяете память в куче, результирующий указатель имеет неограниченное время жизни — его время жизни таково, как долго ваша программа поддерживает его живым.

Основной механизм взаимодействия с кучей в Rust — это тип `Box`. Когда вы пишете `Box::new(value)`, значение помещается в кучу, а то, что вы получаете обратно (`Box<T>`), является указателем на это значение в куче. Когда `Box` в конце концов сбрасывается (dropped), эта память освобождается.

Если вы забудете освободить память кучи, она будет торчать там вечно, и ваше приложение в конце концов съест всю память на вашей машине. Это называется *утечкой памяти* (leaking memory) и обычно является тем, чего вы хотите избежать. Однако существуют случаи, когда вы явно хотите утечь память. Например, скажем, у вас есть конфигурация только для чтения, к

которой должна иметь доступ вся программа. Вы можете выделить её в куче и явно утечь её с помощью `Box::leak`, чтобы получить ссылку с временем жизни `'static`.

Статическая память

Статическая память (static memory) — это на самом деле собирательный термин для нескольких тесно связанных областей, расположенных в файле, в который компилируется ваша программа. Эти области автоматически загружаются в память вашей программы, когда эта программа выполняется. Значения в статической памяти живут на протяжении всего выполнения вашей программы. Статическая память вашей программы содержит бинарный код программы, который обычно отображается как доступный только для чтения. Когда ваша программа выполняется, она проходит через бинарный код инструкция за инструкцией и перепрыгивает по нему всякий раз, когда вызывается функция. Статическая память также содержит память для переменных, которые вы объявляете с ключевым словом `static`, а также определённые константные значения в вашем коде, например строки.

Специальное время жизни `'static`, которое получает своё название от области статической памяти, помечает ссылку как действительную «так долго, как существует статическая память», то есть до тех пор, пока программа не завершится. Поскольку память статической переменной выделяется при запуске программы, ссылка на переменную в статической памяти по определению является `'static`, так как она не освобождается до завершения программы. Обратное неверно — могут существовать ссылки `'static`, которые не указывают на статическую память, — но название всё равно уместно: как только вы создаёте ссылку со статическим временем жизни, то, на что она указывает, с точки зрения остальной программы с тем же успехом может быть в статической памяти, поскольку её можно использовать сколь угодно долго, как пожелает ваша программа.

Вы будете встречать время жизни `'static` гораздо чаще, чем по-настоящему статическую память (через ключевое слово `static`, например), при работе с Rust. Это потому, что `'static` часто появляется в ограничениях (bounds) на параметрах типов. Ограничение вида `T: 'static` указывает, что параметр типа `T` способен жить столь долго, сколько мы его сохраняем, вплоть до и включая оставшееся время выполнения программы. По сути, это ограничение требует, чтобы `T` был владеющим (owned) и самодостаточным, либо в том смысле, что он

не заимствует другие (не-`'static`) значения, либо в том, что всё, что он заимствует, тоже является `'static` и, таким образом, будет существовать до конца программы. Хорошим примером `'static` в качестве ограничения является функция `std::thread::spawn`, которая создаёт новый поток и требует, чтобы передаваемое ей замыкание было `'static`. Поскольку новый поток может пережить текущий поток, новый поток не может ссылаться ни на что, хранящееся в стеке старого потока. Новый поток может ссылаться только на значения, которые будут жить на протяжении всего его собственного времени жизни, которое может быть всем оставшимся временем выполнения программы.

ПРИМЕЧАНИЕ Вы можете задаться вопросом, чем `const` отличается от `static`. Ключевое слово `const` объявляет следующий за ним элемент константой. Константы могут быть полностью вычислены во время компиляции, и любой код, который на них ссылается, заменяется вычисленным значением константы во время компиляции. У константы нет памяти или другого хранилища, связанного с ней (она не является местом). Можно думать о константе как об удобном имени для конкретного значения.

Владение (Ownership)

Модель памяти Rust центрируется вокруг идеи, что у всех значений есть единственный *владелец* (owner) — то есть ровно одна локация (обычно область видимости) ответственна за окончательное освобождение каждого значения. Это обеспечивается через borrow checker. Если значение перемещено (moved), например путём присваивания его новой переменной, помещения в вектор или размещения в куче, владение значением переходит от старой локации к новой. С этого момента вы больше не можете обращаться к значению через переменные, исходящие из исходного владельца, даже хотя биты, составляющие значение, технически всё ещё там. Вместо этого вы должны обращаться к перемещённому значению через переменные, которые ссылаются на его новую локацию.

Некоторые типы — бунтари и не следуют этому правилу. Если тип значения реализует специальный трейт `Copу`, значение не считается перемещённым, даже если оно переприсвоено новой локации памяти. Вместо этого значение *копируется* (copied), и обе — старая и новая — локации остаются доступными. По сути, ещё один идентичный экземпляр того же значения создаётся в месте назначения перемещения.

Большинство примитивных типов в Rust, таких как целочисленные типы и типы с плавающей точкой, являются `Copu`. Чтобы быть `Copu`, должно быть возможно дублировать значения типа, просто скопировав его биты. Это исключает все типы, которые *содержат* не-`Copu` типы, а также любой тип, который владеет ресурсом, который он должен освободить при сбросе значения.

Чтобы понять, почему, рассмотрим, что произошло бы, если бы тип вроде `Box` был `Copu`. Если бы мы выполнили `box2 = box1`, то и `box1`, и `box2` оба полагали бы, что они владеют памятью кучи, выделенной для `box`, и они оба попытались бы освободить её, когда выйдут из области видимости. Освобождение памяти дважды могло бы иметь катастрофические последствия.

Когда у значения больше нет применения, ответственность владельца — выполнить любую необходимую очистку для этого значения путём его *сброса* (dropping). В Rust сброс происходит автоматически, когда переменная, хранящая значение, выходит из области видимости. Типы обычно рекурсивно сбрасывают содержащиеся в них значения, так что сброс переменной сложного типа может привести к сбросу многих значений. Благодаря дискретной модели владения Rust мы не можем случайно сбросить одно и то же значение несколько раз. Переменная, хранящая ссылку на другое значение, не владеет этим другим значением, поэтому значение не сбрасывается, когда переменная сбрасывается.

Код в листинге 2-3 даёт краткое резюме правил, касающихся владения, семантики перемещения и копирования, и сброса.

```
let x1 = 42;
let y1 = Box::new(84);
{ // начинает новую область видимости
❶ let z = (x1, y1);
    // z выходит из области видимости и сбрасывается;
    // это, в свою очередь, сбрасывает значения из x1 и y1
❷ }
// значение x1 является Copu, поэтому оно не было перемещено в z
❸ let x2 = x1;
// значение y1 не Copu, поэтому оно было перемещено в z
❹ // let y2 = y1;
```

Листинг 2-3: Семантика перемещения и копирования

Мы начинаем с двух значений: числа 42 и `Box` (значение, выделенное в куче), содержащего число 84. Первое является `Copu`, тогда как второе — нет. Когда мы

помещаем `x1` и `y1` в кортеж `z` ❶, `x1` копируется в `z`, тогда как `y1` перемещается в `z`. С этого момента `x1` продолжает быть доступным и может быть снова использован ❸. С другой стороны, `y1` становится недоступным, как только его значение было перемещено ❹, и любая попытка обратиться к нему вызвала бы ошибку компиляции. Когда `z` выходит из области видимости ❷, кортежное значение, которое он содержит, сбрасывается, и это, в свою очередь, сбрасывает значение, скопированное из `x1`, и значение, перемещённое из `y1`. Когда `Box` из `y1` сбрасывается, он также освобождает память кучи, использованную для хранения значения `y1`.

ПОРЯДОК СБРОСА (DROP ORDER)

Rust автоматически сбрасывает значения, когда они выходят из области видимости, например `x1` и `y1` во внутренней области видимости в листинге 2-3. Правила порядка, в котором происходит сброс, довольно просты: переменные (включая аргументы функций) сбрасываются в обратном порядке, а вложенные значения сбрасываются в порядке исходного кода.

Поначалу это может звучать странно — почему такое расхождение? Если мы внимательно присмотримся, в этом есть много смысла. Скажем, вы пишете функцию, которая объявляет строку, а затем вставляет ссылку на эту строку в новую хеш-таблицу. Когда функция возвращается, хеш-таблица должна быть сброшена первой; если бы строка была сброшена первой, хеш-таблица содержала бы недопустимую ссылку! В общем случае более поздние переменные могут содержать ссылки на более ранние значения, тогда как обратное не может произойти из-за правил времён жизни Rust. И по этой причине Rust сбрасывает переменные в обратном порядке.

Теперь мы могли бы иметь то же самое поведение для вложенных значений, например значений в кортеже, массиве или структуре, но это, вероятно, удивило бы пользователей. Если бы вы сконструировали массив, содержащий два значения, показалось бы странным, если бы последний элемент массива сбрасывался первым. То же самое применимо к кортежам и структурам, где наиболее интуитивное поведение — чтобы первый элемент кортежа или поле сбрасывался первым, затем второй и так далее. В отличие от переменных, нет необходимости обращать порядок сброса в этом случае, поскольку Rust (в настоящее время) не позволяет самоссылки в одном значении. Так что Rust выбирает интуитивный вариант.

Заимствование и времена жизни (Borrowing and Lifetimes)

Rust позволяет владельцу значения одолжить (lend out) это значение другим, не отдавая владение, через ссылки (references). *Ссылки* (References) — это указатели, которые приходят с дополнительным контрактом о том, как они могут использоваться, например предоставляет ли ссылка эксклюзивный доступ к значению, на которое она ссылается, или может ли значение, на которое она ссылается, также иметь другие ссылки, указывающие на него.

Разделяемые ссылки (Shared References)

Разделяемая ссылка, `&T`, как следует из названия, — это указатель, который может быть разделён. Любое количество других ссылок может существовать на одно и то же значение, и каждая разделяемая ссылка является `Copy`, так что вы можете тривиально создавать их больше. Значения за разделяемыми ссылками неизменяемы; вы не можете изменять или переприсваивать значение, на которое указывает разделяемая ссылка, ни приводить (cast) разделяемую ссылку к изменяемой.

Компилятору Rust разрешено предполагать, что значение, на которое указывает разделяемая ссылка, *не изменится*, пока эта ссылка живёт. Например, если компилятор Rust видит, что значение за разделяемой ссылкой читается несколько раз в функции, он вправе прочитать его только один раз и переиспользовать это значение. Конкретнее, утверждение в листинге 2-4 никогда не должно провалиться.

```
fn cache(input: &i32, sum: &mut i32) {  
    *sum = *input + *input;  
    assert_eq!(*sum, 2 * *input);  
}
```

Листинг 2-4: Rust предполагает, что разделяемые ссылки неизменяемы.

Применяет ли компилятор данную оптимизацию или нет — более или менее неважно. Эвристики компилятора меняются со временем, поэтому вы обычно хотите писать код, ориентируясь на то, что компилятору *разрешено* делать, а не на то, что он на самом деле делает в конкретном случае в конкретный момент времени.

Изменяемые ссылки (Mutable References)

Альтернатива разделяемой ссылке — это изменяемая ссылка: `&mut T`. С изменяемыми ссылками компилятору Rust снова разрешено в полной мере использовать контракт, с которым приходит эта ссылка: компилятор предполагает, что нет других потоков, обращающихся к целевому значению, ни через разделяемую, ни через изменяемую ссылку. Иными словами, он предполагает, что изменяемая ссылка *эксклюзивна* (exclusive). Это позволяет некоторые интересные оптимизации, которые не так легко доступны в других языках. Возьмём, например, код в листинге 2-5.

```
fn noalias(input: &i32, output: &mut i32) {  
    if *input == 1 {  
        ❶ *output = 2;  
    }  
    ❷ if *input != 1 {  
        *output = 3;  
    }  
}
```

Листинг 2-5: Rust предполагает, что изменяемые ссылки эксклюзивны.

В Rust компилятор может предположить, что `input` и `output` не указывают на одну и ту же память. Следовательно, переписывание `output` на ❶ не может повлиять на проверку на ❷, и вся функция может быть скомпилирована как единый блок `if-else`. Если бы компилятор не мог полагаться на контракт эксклюзивной изменяемости, эта оптимизация была бы недопустима, поскольку `input` со значением 1 мог бы тогда привести к `output` равному 3 в случае вроде `noalias(&x, &mut x)`.

Изменяемая ссылка позволяет изменять только ту локацию памяти, на которую ссылка указывает. Можете ли вы изменять значения, лежащие за пределами непосредственной ссылки, зависит от методов, предоставляемых типом, который лежит между ними. Это может быть легче понять на примере, поэтому рассмотрим листинг 2-6.

```
let x = 42;  
let mut y = &x; // y имеет тип &i32  
let z = &mut y; // z имеет тип &mut &i32
```

Листинг 2-6: Изменяемость применяется только к непосредственно адресуемой памяти.

В этом примере вы можете изменить значение указателя `y` на другое значение (то есть другой указатель), заставив его ссылаться на другую переменную, но вы не можете изменить значение, на которое указывается (то есть значение `x`). Аналогично, вы можете изменить значение указателя `y` через `z`, но вы не можете изменить сам `z`, чтобы он хранил другую ссылку.

Главное отличие между владением значением и наличием изменяемой ссылки на него состоит в том, что владелец ответственен за сброс значения, когда оно больше не нужно. Помимо этого, вы можете делать через изменяемую ссылку всё, что можете делать, если владеете значением, с одной оговоркой: если вы перемещаете значение из-за изменяемой ссылки, то вы должны оставить на его месте другое значение. Если бы вы этого не сделали, владелец всё равно полагал бы, что ему нужно сбросить значение, но не было бы значения, которое можно сбросить!

Листинг 2-7 даёт пример способов, которыми вы можете перемещать значение из-за изменяемой ссылки.

```
fn replace_with_84(s: &mut Box<i32>) {  
    // это нельзя, так как *s оказалось бы пустым:  
    ❶ // let was = *s;  
    // но вот это можно:  
    ❷ let was = std::mem::take(s);  
    // и вот это тоже:  
    ❸ *s = was;  
    // мы можем обменять значения за &mut:  
    let mut r = Box::new(84);  
    ❹ std::mem::swap(s, &mut r);  
    assert_ne!(*r, 84);  
}  
let mut s = Box::new(42);  
replace_with_84(&mut s);  
❺
```

Листинг 2-7: Доступ через изменяемую ссылку должен оставить значение позади.

Я добавил закомментированные строки, представляющие недопустимые операции. Вы не можете просто переместить значение наружу ❶, поскольку вызывающий по-прежнему считал бы, что владеет этим значением, и снова освободил бы его на ❺, что привело бы к двойному освобождению (double free). Если вы просто хотите оставить позади некоторое допустимое значение, `std::mem::take` ❷ — хороший кандидат. Он эквивалентен `std::mem::replace(&mut value, Default::default())`; он перемещает значение из-за изменяемой ссылки,

но оставляет на его месте новое, заданное по умолчанию значение для типа. Значение по умолчанию — это отдельное, владеющее значение, поэтому вызывающий безопасно сбросит его, когда область видимости закончится на ⑤.

Альтернативно, если вам не нужно старое значение за ссылкой, вы можете перезаписать его значением, которым вы уже владеете ③, оставив вызывающему сбросить значение позже. Когда вы это делаете, значение, которое раньше было за изменяемой ссылкой, сбрасывается немедленно.

Наконец, если у вас есть две изменяемые ссылки, вы можете обменять их значения, не владея ни одним из них ④, поскольку обе ссылки в итоге окажутся с допустимым владеемым значением для их владельцев, чтобы в конце концов их освободить.

Внутренняя изменяемость (Interior Mutability)

Некоторые типы предоставляют *внутреннюю изменяемость* (interior mutability), что означает, что они позволяют вам изменять значение через разделяемую ссылку. Эти типы обычно полагаются на дополнительные механизмы (вроде атомарных инструкций процессора) или инварианты, чтобы обеспечить безопасную изменяемость, не полагаясь на семантику эксклюзивных ссылок. Они обычно делятся на две категории: те, что позволяют получить изменяемую ссылку через разделяемую ссылку, и те, что позволяют заменить значение, имея только разделяемую ссылку.

Первая категория состоит из типов вроде `Mutex` и `RefCell`, которые содержат механизмы безопасности, чтобы гарантировать, что для любого значения, на которое они дают изменяемую ссылку, может одновременно существовать только одна изменяемая ссылка (и никаких разделяемых ссылок). Под капотом эти типы (и подобные им) полагаются на тип под названием `UnsafeCell`, чьё название должно немедленно заставить вас колебаться, прежде чем его использовать. Мы рассмотрим `UnsafeCell` подробнее в главе 10, но пока вам следует знать только то, что это *единственный* корректный способ изменять что-либо через разделяемую ссылку.

Другие категории типов, предоставляющих внутреннюю изменяемость, — это те, что не выдают изменяемую ссылку на внутреннее значение, а вместо этого просто дают вам методы для манипулирования значением на месте. Атомарные целочисленные типы в `std::sync::atomic` и тип `std::cell::Cell`

попадают в эту категорию. Вы не можете получить ссылку напрямую на `usize` или `i32`, лежащий за таким типом, но вы можете прочитать и заменить его значение в данной точке времени.

ПРИМЕЧАНИЕ Тип `Cell` в стандартной библиотеке — это интересный пример безопасной внутренней изменяемости через инварианты. Он не является разделяемым между потоками и никогда не выдаёт ссылку на значение, содержащееся в `Cell`. Вместо этого все методы либо полностью заменяют значение, либо возвращают копию содержащегося значения. Поскольку никаких ссылок на внутреннее значение существовать не может, всегда допустимо его перемещать. А поскольку `Cell` не разделяется между потоками, внутреннее значение никогда не будет конкурентно изменено, даже хотя изменение происходит через разделяемую ссылку.

Времена жизни (Lifetimes)

Если вы читаете эту книгу, вы, вероятно, уже знакомы с концепцией времён жизни, скорее всего через повторяющиеся уведомления от компилятора о нарушениях правил времён жизни. Этого уровня понимания вам хватит для большинства кода на Rust, который вы будете писать, но по мере того как мы глубже погружаемся в более сложные части Rust, вам понадобится более строгая ментальная модель, с которой можно работать.

Новых разработчиков Rust часто учат думать о временах жизни как о соответствующих областям видимости: время жизни начинается, когда вы берёте ссылку на некоторую переменную, и заканчивается, когда эта переменная перемещается или выходит из области видимости. Это часто верно и обычно полезно, но реальность немного сложнее. *Время жизни* (lifetime) — это на самом деле имя для области кода, на протяжении которой некоторая ссылка должна быть действительной. Хотя время жизни часто будет совпадать с областью видимости, оно не обязано, как мы увидим позже в этом разделе.

Времена жизни и borrow checker

В сердце времён жизни Rust находится *borrow checker*. Всякий раз, когда используется ссылка с некоторым временем жизни `'a`, borrow checker проверяет, что `'a` всё ещё *живо* (alive). Он делает это, отслеживая путь обратно к месту, где `'a` начинается — где была взята ссылка, — от точки использования и проверяя, что на этом пути нет конфликтующих использований. Это

гарантирует, что ссылка по-прежнему указывает на значение, к которому безопасно обращаться. Это похоже на высокоуровневую ментальную модель «потока данных» (data flow), которую мы обсуждали ранее в этой главе; компилятор проверяет, что поток ссылки, к которой мы обращаемся, не конфликтует ни с какими другими параллельными потоками.

Листинг 2-8 показывает простой пример кода с аннотациями времён жизни для ссылки на `x`.

```
let mut x = Box::new(42);
❶ let r = &x;           // 'a
if rand() > 0.5 {
❷ *x = 84;
} else {
❸ println!("{}", r);    // 'a
}
❹
```

Листинг 2-8: Времена жизни не обязаны быть непрерывными.

Время жизни начинается на ❶, когда мы берём ссылку на `x`. В первой ветке ❷ мы затем немедленно пытаемся изменить `x`, меняя его значение на `84`, что требует `&mut x`. Borrow checker берёт изменяемую ссылку на `x` и немедленно проверяет её использование. Он не находит конфликтующих использований между моментом, когда ссылка была взята, и моментом её использования, поэтому принимает код. Это может стать сюрпризом, если вы привыкли думать о временах жизни как об областях видимости, поскольку `r` всё ещё находится в области видимости на ❷ (он выходит из области видимости на ❸). Но borrow checker достаточно умен, чтобы понять, что `r` никогда не используется позже, если выбрана эта ветка, и поэтому нормально, что к `x` здесь обращаются изменяемо. Или, иначе говоря, время жизни, созданное на ❶, не простирается в эту ветку: нет потока от `r` за пределами ❷, и поэтому нет конфликтующих потоков. Затем borrow checker находит использование `r` в выражении печати на ❸. Он проходит путь обратно к ❶ и не находит конфликтующих использований (❷ не на этом пути), так что он принимает и это использование.

Если бы мы добавили ещё одно использование `r` на ❹ в листинге 2-8, код больше не скомпилировался бы. Время жизни `'a` тогда длилось бы от ❶ вплоть до ❹ (последнее использование `r`), и когда borrow checker проверил бы наше новое использование `r`, он обнаружил бы конфликтующее использование на ❷.

Времена жизни могут стать довольно запутанными. В листинге 2-9 вы можете

увидеть пример времени жизни, у которого есть *дыры* (holes), где оно периодически недействительно между местом начала и местом окончательного завершения.

```
let mut x = Box::new(42);
❶ let mut z = &x;           // 'a
for i in 0..100 {
  ❷ println!("{}", z);      // 'a
  ❸ x = Box::new(i);
  ❹ z = &x;                 // 'a
}
println!("{}", z);          // 'a
```

Листинг 2-9: Времена жизни могут иметь дыры.

Время жизни начинается на ❶, когда мы берём ссылку на `x`. Затем мы перемещаем `x` на ❸, что заканчивает время жизни `'a`, поскольку оно больше недействительно. Borrow checker принимает это перемещение, считая `'a` закончившимся на ❷, что не оставляет конфликтующих потоков от `x` на ❸. Затем мы перезапускаем время жизни, обновляя ссылку в `z` на ❹. Независимо от того, закидывается ли теперь код обратно к ❷ или продолжается к финальному выражению печати, оба этих использования теперь имеют допустимое значение, от которого можно проводить поток, и нет конфликтующих потоков, так что borrow checker принимает код!

Опять же, это идеально соответствует модели памяти «потока данных», которую мы обсуждали ранее. Когда `x` перемещается, `z` перестаёт существовать. Когда мы позже переприсваиваем `z`, мы создаём совершенно новую переменную, которая существует только с этого момента вперёд. С этой моделью в уме этот пример не выглядит странным.

Обобщённые времена жизни (Generic Lifetimes)

Иногда вам нужно хранить ссылки внутри своих собственных типов. Эти ссылки должны иметь время жизни, чтобы borrow checker мог проверять их действительность, когда они используются в различных методах этого типа. Это особенно верно, если вы хотите, чтобы метод вашего типа возвращал ссылку, которая переживает ссылку на `self`.

Rust позволяет сделать определение типа обобщённым (generic) по одному или нескольким временам жизни, точно так же как он позволяет сделать его обобщённым по типам. Книга «The Rust Programming Language» Стива

Клабника и Кэрол Николс (No Starch Press, 2018) рассматривает эту тему в некоторых деталях, поэтому я не буду повторять основы здесь. Но по мере того как вы пишете более сложные типы такого рода, есть две тонкости во взаимодействии между такими типами и временами жизни, о которых вам следует знать.

Во-первых, если ваш тип также реализует `Drop`, то сброс вашего типа считается использованием любого времени жизни или типа, по которому ваш тип обобщён. По сути, когда экземпляр вашего типа сбрасывается, borrow checker проверит, что по-прежнему допустимо использовать любые обобщённые времена жизни вашего типа, прежде чем сбросить его. Это необходимо на случай, если ваш код сброса *действительно* использует какие-либо из этих ссылок. Если ваш тип не реализует `Drop`, сброс типа *не* считается использованием, и пользователи вольны делать недействительными любые ссылки, хранящиеся в вашем типе, до тех пор, пока они больше его не используют, как мы видели в листинге 2-7. Мы поговорим подробнее об этих правилах сброса в главе 10.

Во-вторых, хотя тип может быть обобщён по нескольким временам жизни, делать так часто служит лишь к ненужному усложнению сигнатуры вашего типа. Обычно типу достаточно быть обобщённым по одному времени жизни, и компилятор будет использовать более короткое из времён жизни для любых ссылок, вставленных в ваш тип, в качестве этого единого времени жизни. Вам действительно стоит использовать несколько обобщённых параметров времени жизни, только если у вас есть тип, содержащий несколько ссылок, и его методы возвращают ссылки, которые должны быть привязаны к времени жизни только *одной* из этих ссылок.

Рассмотрите тип в листинге 2-10, который даёт вам итератор по частям строки, разделённым некоторой другой строкой.

```
struct StrSplit<'s, 'p> {
    delimiter: &'p str,
    document: &'s str,
}
impl<'s, 'p> Iterator for StrSplit<'s, 'p> {
    type Item = &'s str;
    fn next(&self) -> Option<Self::Item> {
        todo!()
    }
}
fn str_before(s: &str, c: char) -> Option<&str> {
    StrSplit { document: s, delimiter: &c.to_string() }.next()
}
```

```
}
```

Листинг 2-10: Тип, который должен быть обобщён по нескольким временам жизни

Когда вы конструируете этот тип, вам нужно задать `delimiter` и `document` для поиска, оба из которых являются ссылками на строковые значения. Когда вы запрашиваете следующую строку, вы получаете ссылку в `document`. Рассмотрите, что произошло бы, если бы вы использовали единственное время жизни в этом типе. Значения, выдаваемые итератором, были бы привязаны к времени жизни `document` и `delimiter`. Это сделало бы `str_before` невозможной для написания: возвращаемый тип имел бы время жизни, связанное с переменной, локальной для функции — `String`, производимым `to_string`, — и borrow checker отверг бы код.

Вариантность времён жизни (Lifetime Variance)

Вариантность (variance) — это концепция, с которой программисты часто сталкиваются, но редко знают её название, потому что она в основном невидима. На первый взгляд, вариантность описывает, какие типы являются подтипами (subtypes) других типов и когда подтип можно использовать вместо супертипа (supertype) (и наоборот). В широком смысле, тип `A` является подтипом другого типа `B`, если `A` по крайней мере столь же полезен, как `B`. Вариантность — это причина, по которой в Java вы можете передать `Turtle` в функцию, принимающую некий неуточнённый `Animal`, если `Turtle` является подтипом `Animal`, или почему в Rust вы можете передать `&'static str` в функцию, которая принимает `&'a str`.

Хотя вариантность обычно скрывается из виду, она возникает достаточно часто, чтобы нам нужно было иметь рабочее знание о ней. `Turtle` является подтипом `Animal`, поскольку `Turtle` более «полезна», чем некоторый неуточнённый `Animal` — `Turtle` может делать всё, что может `Animal`, и, вероятно, больше. Аналогично, `'static` является подтипом `'a`, поскольку `'static` живёт по крайней мере столь же долго, как любое `'a`, и поэтому более полезно. Или, в более общем виде, если `'b: 'a` (`'b` переживает `'a`), то `'b` является подтипом `'a`. Это, очевидно, не формальное определение, но оно достаточно близко для практического использования.

У всех типов есть вариантность, которая определяет, какие другие похожие

типы можно использовать вместо этого типа. Существует три вида вариантности: ковариантность (covariant), инвариантность (invariant) и контравариантность (contravariant). Тип *ковариантен* (covariant), если вы можете просто использовать подтип вместо самого типа. Например, если переменная имеет тип `&'a T`, вы можете предоставить ей значение типа `&'static T`, поскольку `&'a T` ковариантен в `'a`. `&'a T` также ковариантен в `T`, так что вы можете передать `&Vec<&'static str>` в функцию, которая принимает `&Vec<&'a str>`.

Некоторые типы *инвариантны* (invariant), что означает, что вы должны предоставить ровно заданный тип. `&mut T` является примером этого — если функция принимает `&mut Vec<&'a str>`, вы не можете передать ей `&mut Vec<&'static str>`. То есть `&mut T` инвариантен в `T`. Если бы вы могли, функция могла бы поместить недолговечную строку внутрь `Vec`, который вызывающий затем продолжил бы использовать, думая, что это `Vec<&'static str>` и, таким образом, что содержащаяся строка является `'static`! Любой тип, обеспечивающий изменяемость, в общем случае инвариантен по той же причине — например, `Cell<T>` инвариантен в `T`.

Последняя категория, *контравариантность* (contravariance), возникает для аргументов функций. Функциональные типы более полезны, если они нормально относятся к тому, что их аргументы *менее* полезны. Это яснее, если сравнить вариантность самих типов аргументов с их вариантностью при использовании в качестве аргументов функций:

```
let x: &'static str; // более полезно, живёт дольше
let x: &'a          str; // менее полезно, живёт меньше

fn take_func1(&'static str) // строже, поэтому менее полезно
fn take_func2(&'a str)      // менее строго, более полезно
```

Это перевёрнутое отношение указывает, что `Fn(T)` контравариантен в `T`.

Так зачем же вам нужно узнавать о вариантности, когда дело доходит до времён жизни? Вариантность становится релевантной, когда вы рассматриваете, как обобщённые параметры времён жизни взаимодействуют с borrow checker. Рассмотрите тип вроде показанного в листинге 2-11, который использует несколько времён жизни в одном поле.

```
struct MutStr<'a, 'b> {
    s: &'a mut &'b str
```

```
}  
let mut s = "hello";  
❶ *MutStr { s: &mut s }.s = "world";  
println!("{}", s);
```

Листинг 2-11: Тип, который должен быть обобщён по нескольким временам жизни

На первый взгляд использование двух времён жизни здесь кажется ненужным — у нас нет методов, которым нужно различать заимствование разных частей структуры, как у нас было в `StrSplit` в листинге 2-10. Но если вы замените два времени жизни здесь единственным `'a`, код больше не скомпилируется! И всё это из-за вариантности.

На ❶ компилятор должен определить, на какое время жизни должен быть установлен параметр(ы) времени жизни. Если есть два времени жизни, `'a` устанавливается на ещё-не-определённое время жизни заимствования `s`, а `'b` устанавливается на `'static`, поскольку это время жизни предоставленной строки `"hello"`. Если есть только одно время жизни `'a`, компилятор делает вывод, что это время жизни должно быть `'static`.

Когда мы позже попытаемся обратиться к строковой ссылке `s` через разделяемую ссылку, чтобы её напечатать, компилятор пытается сократить изменяемое заимствование `s`, используемое `MutStr`, чтобы позволить разделяемое заимствование `s`.

В случае двух времён жизни `'a` просто заканчивается прямо перед `println`, а `'b` остаётся прежним. В случае единственного времени жизни, с другой стороны, мы сталкиваемся с проблемами. Компилятор хочет сократить заимствование `s`, но чтобы это сделать, ему также пришлось бы сократить заимствование `str`. Хотя `&'static str` в общем случае можно сократить до любого `&'a str` (`&'a T` ковариантен в `'a`), здесь это находится за `&mut T`, который инвариантен в `T`. Инвариантность требует, чтобы соответствующий тип никогда не заменялся под- или супертипом, так что попытка компилятора сократить заимствование проваливается, и он сообщает, что список всё ещё изменяемо заимствован. Ой!

Из-за сниженной гибкости, налагаемой инвариантностью, вы хотите гарантировать, что ваши типы остаются ковариантными (или контравариантными, где это уместно) по как можно большему числу их обобщённых параметров. Если это требует введения дополнительных аргументов времени жизни, вам нужно тщательно взвесить когнитивную

стоимость добавления ещё одного параметра против эргономической стоимости инвариантности.

Итоги (Summary)

Цель этой главы состояла в том, чтобы установить прочный, общий фундамент, на котором мы можем строить в последующих главах. К настоящему моменту, я надеюсь, вы чувствуете, что у вас есть твёрдое понимание модели памяти и владения Rust, и что те ошибки, которые вы, возможно, получали от borrow checker, кажутся менее загадочными. Возможно, вы уже знали отдельные кусочки того, что мы здесь рассмотрели, но, надеюсь, эта глава дала вам более целостный образ того, как всё это сочетается друг с другом. В следующей главе мы сделаем нечто похожее для типов. Мы пройдемся по тому, как типы представлены в памяти, посмотрим, как обобщения (generics) и трейты (traits) производят выполняемый код, и взглянем на некоторые специальные конструкции типов и трейтов, которые Rust предлагает для более продвинутых сценариев использования.